UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA, MATEMÁTICA, Y CIENCIAS FÍSICAS



PROYECTO FINAL ROBOSCRIPT

José Alfredo Garcia Rodríguez 3490-24-13232

José Ricardo Martínez García 3490-20-19500

Ingeniería en Sistemas y Ciencias de la Computación

AUTÓMATAS Y LENGUAJES FORMALES

Ing. Mario Roberto Fuentes Orozco

Octubre de 2026.

Contenido

1. Introducción 1
    1.1 Herramientas utilizadas 3
    1.2 Repositorio 3
2. Descripción de RoboScript 3
    2.1 Propósito 3
    2.2 Estructura de un programa 4
    2.3 Instrucciones del lenguaje 4
3. Lista de tokens 5
    3.1 Palabras reservadas 5
    3.2 Tokens con formato definido 6
    3.3 Tokens para formatos incorrectos 6
    3.4 Identificadores, números y otros elementos 6
4. Expresiones regulares 7
    4.1 Reglas de los tokens 7
    4.2 Prioridad entre reglas 8
5. Gramática del lenguaje 9
    5.1 Notaciones usadas 9
    5.2 Gramática en EBNF 9
    5.3 Gramática en BNF 10
    5.4 Propiedades de la gramática 10
    5.5 Gramática en ANTLR 11
6. Arquitectura y ejecución 13
    6.1 Estructura del proyecto 13
    6.2 Flujo de ejecución 14
    6.3 Compilación y ejecución 14
7. Analizador léxico 15
    7.1 Funcionamiento del lexer 15
    7.2 Manejo de errores léxicos 15
    7.3 Tabla de tokens 16
8. Analizador sintáctico 16
    8.1 Funcionamiento del parser 16
    8.2 Errores detectados por el parser 16
    8.3 Recorrido del árbol y ResumenListener 17
    8.4 Reporte final 18
9. Archivos de prueba 18
    9.1 Archivos válidos 19
    9.2 Archivos inválidos 22
10. Ejemplos de errores léxicos y sintácticos 23
    10.1 Errores léxicos 23
    10.2 Errores sintácticos 24
    10.3 Errores léxicos y sintácticos en un mismo archivo 25
    10.4 Resumen de mensajes 25
11. Capturas de pantalla 26
12. Alcance y decisiones de diseño 29
13. Conclusiones 29
Anexo A. Casos de prueba extensos 30
    A.1 extremo_valido_almacen.robot 30
    A.2 extremo_invalido_errores.robot 32

1. Introducción

Este documento describe el analizador léxico y sintáctico que desarrollamos para RoboScript. El programa lee un archivo con extensión .robot, lo convierte en tokens, verifica que las instrucciones sigan la gramática del lenguaje y muestra un reporte con los errores encontrados y un resumen del programa analizado.

Nos concentramos en el análisis del lenguaje: reconocer sus elementos léxicos, validar su estructura y reportar los errores con su línea y, cuando corresponde, su columna.

1.1 Herramientas utilizadas

Tabla 1. Herramientas del proyecto

Herramienta	Versión	Uso en el proyecto
Java (JDK)	21	Lenguaje de implementación.
ANTLR	4.13.2	Generación del lexer y del parser a partir de RoboScript.g4.
Apache Maven	3.9 o superior	Compilación, generación del código de ANTLR y empaquetado.
maven-shade-plugin	3.5.1	Genera roboscript.jar, un jar ejecutable que incluye el runtime de ANTLR.
IntelliJ IDEA Community	-	Entorno de desarrollo y ejecución de pruebas.
Git y GitHub	-	Control de versiones y entrega del código fuente.

1.2 Repositorio

El código fuente y los archivos de prueba están en el repositorio de GitHub del proyecto:

https://github.com/Alfre-0/ProyectoFinalAutomatas-RoboScript.git

2. Descripción de RoboScript

2.1 Propósito

RoboScript es un lenguaje sencillo para escribir las instrucciones que debe ejecutar un robot dentro de un entorno simulado. Un programa indica qué robot lo ejecuta, en qué coordenada empieza y la lista de acciones que debe realizar: avanzar y retroceder, girar, seguir una línea durante un tiempo, levantar y soltar objetos, cambiar la velocidad y moverse hacia una coordenada. También permite repetir un grupo de instrucciones y ejecutar instrucciones solo cuando el robot encuentra un obstáculo.

En este proyecto el lenguaje solo se analiza, no se ejecuta. El programa comprueba que las instrucciones estén bien escritas y bien organizadas, pero no mueve ningún robot.

2.2 Estructura de un programa

Todo programa tiene dos partes. Primero va el encabezado, formado por la declaración del robot (ROBOT) y la posición inicial (INICIO). Después viene una lista de cero o más instrucciones. Normalmente se escribe una instrucción por línea, aunque el analizador no exige saltos de línea ni sangría. Un comentario empieza con # y llega hasta el final de la línea.

Ejemplo de programa RoboScript

```
ROBOT 123ROB
INICIO X10,Y05

AVANZAR 5
GIRAR DERECHA
LEVANTAR OBJ-0025
MOVER X20,Y15
SOLTAR
SEGUIR LINEA 30s

REPETIR 2
   AVANZAR 10
   GIRAR DERECHA
FIN
```

2.3 Instrucciones del lenguaje

Tabla 2. Instrucciones de RoboScript

Instrucción	Forma	Qué indica	Ejemplo
Declaración del robot	ROBOT CODIGO_ROBOT	Identifica al robot que ejecuta el programa.	ROBOT 123ROB
Posición inicial	INICIO COORDENADA	Coordenada donde empieza el robot.	INICIO X10,Y05
Avanzar	AVANZAR NUMERO	El robot avanza la cantidad indicada.	AVANZAR 5
Retroceder	RETROCEDER NUMERO	El robot retrocede la cantidad indicada.	RETROCEDER 3
Girar	GIRAR DERECHA GIRAR IZQUIERDA	Giro hacia la derecha o hacia la izquierda.	GIRAR DERECHA
Seguir línea	SEGUIR LINEA DURACION	Sigue una línea durante el tiempo indicado.	SEGUIR LINEA 30s
Levantar objeto	LEVANTAR CODIGO_OBJETO	Levanta el objeto indicado.	LEVANTAR OBJ-0025
Mover	MOVER COORDENADA	Se mueve hacia la coordenada indicada.	MOVER X20,Y15
Soltar objeto	SOLTAR	Suelta el objeto que lleva.	SOLTAR
Velocidad	VELOCIDAD PORCENTAJE	Cambia la velocidad del robot al porcentaje indicado.	VELOCIDAD 75%

Instrucción	Forma	Qué indica	Ejemplo
Repetición	REPETIR NUMERO instrucciones FIN	Repite el bloque la cantidad de veces indicada.	REPETIR 3 ... FIN
Condición	SI OBSTACULO instrucciones [SINO instrucciones] FIN	Ejecuta el primer bloque si hay un obstáculo y, si existe SINO, el segundo bloque en caso contrario.	SI OBSTACULO ... FIN

Los bloques REPETIR y SI pueden contener cualquier instrucción, incluidos otros bloques REPETIR y SI. Todo bloque se cierra con FIN.

3. Lista de tokens

El lexer reconoce 31 tipos de token y descarta dos más: los espacios en blanco y los comentarios. En la salida del programa cada token se muestra con el prefijo T_, por ejemplo T_ROBOT o T_CODIGO_OBJETO.

3.1 Palabras reservadas

Tabla 3. Palabras reservadas

Palabra	Token	Uso en el lenguaje
ROBOT	T_ROBOT	Declaración del robot.
INICIO	T_INICIO	Posición inicial.
AVANZAR	T_AVANZAR	Movimiento hacia adelante.
RETROCEDER	T_RETROCEDER	Movimiento hacia atrás.
GIRAR	T_GIRAR	Giro del robot.
DERECHA	T_DERECHA	Dirección de giro.
IZQUIERDA	T_IZQUIERDA	Dirección de giro.
SEGUIR	T_SEGUIR	Primera palabra de SEGUIR LINEA.
LINEA	T_LINEA	Segunda palabra de SEGUIR LINEA.
LEVANTAR	T_LEVANTAR	Levantar un objeto.
MOVER	T_MOVER	Moverse hacia una coordenada.
SOLTAR	T_SOLTAR	Soltar un objeto.
REPETIR	T_REPETIR	Inicio de un bloque de repetición.
SI	T_SI	Inicio de una condición.
SINO	T_SINO	Bloque alternativo de una condición.
FIN	T_FIN	Cierre de los bloques REPETIR y SI.
OBSTACULO	T_OBSTACULO	Condición del SI.

Palabra	Token	Uso en el lenguaje
VELOCIDAD	T_VELOCIDAD	Cambiar la velocidad del robot.

3.2 Tokens con formato definido

Tabla 4. Tokens definidos mediante expresiones regulares

Token	Formato	Ejemplo	Se usa en
CODIGO_ROBOT	Tres dígitos seguidos de tres letras mayúsculas.	123ROB	ROBOT
CODIGO_ZONA	Z- seguido de dos dígitos.	Z-05	Ninguna instrucción actual, añadido para practica
CODIGO_OBJETO	OBJ- seguido de cuatro dígitos.	OBJ-0025	LEVANTAR
COORDENADA	X, dígitos, coma, Y, dígitos.	X10,Y25	INICIO y MOVER
PORCENTAJE	Dígitos seguidos de %.	75%	VELOCIDAD
DURACION	Dígitos seguidos de s (segundos) o m (minutos).	30s, 5m	SEGUIR LINEA

3.3 Tokens para formatos incorrectos

Además de los tokens anteriores, definimos cuatro tokens que reconocen textos que se parecen a un código pero no cumplen su formato. Gracias a ellos el programa puede reportar un mensaje claro, por ejemplo 12ROB no cumple con la expresión regular de CODIGO_ROBOT, en lugar de dividir el texto en tokens sin relación entre sí.

Tabla 5. Tokens de formato incorrecto

Token	Qué reconoce	Ejemplos
CODIGO_ROBOT_INVALIDO	Dígitos seguidos de letras que no cumplen el formato de CODIGO_ROBOT.	12ROB, 123rob, 1234ROB
CODIGO_OBJETO_INVALIDO	OBJ- seguido de una cantidad de dígitos distinta de cuatro.	OBJ-25, OBJ-12345
CODIGO_ZONA_INVALIDO	Z- seguido de una cantidad de dígitos distinta de dos.	Z-5, Z-123
PORCENTAJE_INVALIDO	El símbolo % sin un número delante.	%

3.4 Identificadores, números y otros elementos

Tabla 6. Identificadores, números, operadores y elementos descartados

Token	Formato	Descripción
NUMERO	Uno o más dígitos.	Enteros sin signo. Se usan en AVANZAR, RETROCEDER y REPETIR. En VELOCIDAD solo se acepta para avisar que falta el símbolo %.

Token	Formato	Descripción
ID	Letra o guion bajo, seguido de letras, dígitos o guion bajo.	Palabras que no son reservadas, como ARRIBA. La gramática las usa para detectar una palabra suelta donde se esperaba un código o una dirección.
REL_OP	==, !=, <=, >=, <, >	Operadores relacionales. La gramática actual no los utiliza.
ESPACIOBLANCO	Espacios, tabuladores y saltos de línea.	Se descartan.
COMENTARIO	# hasta el final de la línea.	Se descarta.

Cualquier carácter que no pueda formar parte de estos tokens, como @, $, & o ^, se reporta como error léxico. La coma solo es válida dentro de una COORDENADA.

4. Expresiones regulares

Las expresiones regulares forman parte de RoboScript.g4. ANTLR las usa para generar el lexer. Las tablas de esta sección muestran cada regla como está escrita en ANTLR y su equivalente en la notación habitual de expresiones regulares. En ANTLR, [0-9] es una clase de caracteres, + significa una o más veces, * cero o más veces, las comillas simples encierran texto literal y ~ niega una clase.

4.1 Reglas de los tokens

Tabla 7. Expresiones regulares de los tokens con formato definido

Token	Regla en ANTLR	Equivalente	Ejemplos válidos
CODIGO_ROBOT	[0-9][0-9][0-9][A-Z][A-Z][A-Z]	[0-9]{3}[A-Z]{3}	123ROB, 999XYZ
CODIGO_ZONA	'Z-' [0-9][0-9]	Z-[0-9]{2}	Z-05
CODIGO_OBJETO	'OBJ-' [0-9][0-9][0-9][0-9]	OBJ-[0-9]{4}	OBJ-0025
COORDENADA	'X' [0-9]+ ',' 'Y' [0-9]+	X[0-9]+,Y[0-9]+	X10,Y05 X5,Y5
PORCENTAJE	[0-9]+ '%'	[0-9]+%	75%, 100%
DURACION	[0-9]+ ('s' | 'm')	[0-9]+[sm]	30s, 5m
NUMERO	[0-9]+	[0-9]+	5, 10, 042
ID	[a-zA-Z_] [a-zA-Z_0-9]*	[a-zA-Z_][a-zA-Z_0-9]*	ARRIBA, robot_1
REL_OP	'==' | '!=' | '<=' | '>=' | '<' | '>'	==|!=|<=|>=|<|>	==, <

Tabla 8. Expresiones regulares de formatos incorrectos y elementos descartados

Token	Regla en ANTLR	Equivalente	Ejemplos
CODIGO_ROBOT_INVALIDO	[0-9]+ [A-Za-z]+	[0-9]+[A-Za-z]+	12ROB, 123rob
CODIGO_OBJETO_INVALIDO	'OBJ-' [0-9]+	OBJ-[0-9]+	OBJ-25
CODIGO_ZONA_INVALIDO	'Z-' [0-9]+	Z-[0-9]+	Z-5

Token	Regla en ANTLR	Equivalente	Ejemplos
PORCENTAJE_INVALIDO	'%'	%	%
ESPACIOBLANCO	[\t\r\n]+	[\t\r\n]+	(descartado)
COMENTARIO	'#' ~[\r\n]*	#[^\r\n]*	# texto

4.2 Prioridad entre reglas

Un mismo texto puede cumplir varias reglas a la vez. Por ejemplo, ROBOT cumple la regla de la palabra reservada y también la de ID. ANTLR resuelve estos casos con dos criterios: elige la regla que reconoce más caracteres y, si dos reglas reconocen la misma cantidad, elige la que aparece primero en RoboScript.g4.

Por eso el orden de las reglas en el archivo importa. Primero van las palabras reservadas, después los tokens con formato definido, luego sus versiones incorrectas y al final NUMERO e ID.

Tabla 9. Ejemplos de resolución entre reglas (comprobados con el programa)

Texto	Reglas que coinciden	Token resultante	Motivo
ROBOT	ROBOT, ID	T_ROBOT	Misma longitud. Gana ROBOT, que está definida antes que ID.
SINO	SI, SINO, ID	T_SINO	SI reconoce menos caracteres. SINO e ID empatan y gana SINO por estar primero.
SIGUIENTE	SI, ID	T_ID	ID reconoce más caracteres que SI.
123ROB	CODIGO_ROBOT, CODIGO_ROBOT_INVALIDO	T_CODIGO_ROBOT	Misma longitud. Gana CODIGO_ROBOT por estar primero.
12ROB	CODIGO_ROBOT_INVALIDO	T_CODIGO_ROBOT_INVALIDO	No cumple el formato de CODIGO_ROBOT.
30s	DURACION, CODIGO_ROBOT_INVALIDO	T_DURACION	Misma longitud. Gana DURACION por estar primero.
OBJ-25	CODIGO_OBJETO_INVALIDO	T_CODIGO_OBJETO_INVALIDO	Solo cumple la regla de formato incorrecto.

5. Gramática del lenguaje

5.1 Notaciones usadas

Tabla 10. Notación usada en la gramática

Símbolo	Significado
->	Se define como.
PALABRA	Literal: palabra reservada escrita tal cual, como ROBOT, AVANZAR o FIN.
< >	Obligatorio: cláusula que debe aparecer. Los nombres en minúsculas, como <instruccion>, son reglas de la gramática. Los nombres en mayúsculas, como <NUMERO>, son tokens definidos por una expresión regular.
[ ]	Opcional: la cláusula puede aparecer o no.
{ }	Iteración: la cláusula se repite cero o más veces.
|	Exclusión: se elige una sola de las opciones.

5.2 Gramática en EBNF

Gramática de RoboScript en EBNF

```
<programa>          ->  <declaracion_robot> <declaracion_inicio> { <instruccion> }

<declaracion_robot> ->  ROBOT <CODIGO_ROBOT>
<declaracion_inicio>->  INICIO <COORDENADA>

<instruccion>       ->  <avanzar>
                        | <retroceder>
                        | <girar>
                        | <seguir_linea>
                        | <levantar>
                        | <mover>
                        | <soltar>
                        | <velocidad>
                        | <repetir>
                        | <condicion>

<avanzar>           ->  AVANZAR <NUMERO>
<retroceder>        ->  RETROCEDER <NUMERO>
<girar>             ->  GIRAR <direccion>
<direccion>         ->  DERECHA | IZQUIERDA
<seguir_linea>      ->  SEGUIR LINEA <DURACION>
<levantar>          ->  LEVANTAR <CODIGO_OBJETO>
<mover>             ->  MOVER <COORDENADA>
<soltar>            ->  SOLTAR
<velocidad>         ->  VELOCIDAD <PORCENTAJE>

<repetir>           ->  REPETIR <NUMERO> { <instruccion> } FIN
<condicion>         ->  SI OBSTACULO { <instruccion> }
                        [ SINO { <instruccion> } ] FIN
```

5.3 Gramática en BNF

La misma gramática escrita en BNF, sin las cláusulas opcionales [ ] ni las iteraciones { }. En su lugar se usan la recursividad (<lista_instrucciones>) y ε, que representa la cadena vacía. El resto de la notación es la de la sección 5.1.

Gramática de RoboScript en BNF

```
<programa>          ->  <declaracion_robot> <declaracion_inicio> <lista_instrucciones>

<declaracion_robot> ->  ROBOT <CODIGO_ROBOT>
<declaracion_inicio>->  INICIO <COORDENADA>

<lista_instrucciones> -> <instruccion> <lista_instrucciones>
                        | ε

<instruccion>       ->  <avanzar> | <retroceder> | <girar> | <seguir_linea>
                        | <levantar> | <mover> | <soltar> | <velocidad> | <repetir>
                        | <condicion>

<avanzar>           ->  AVANZAR <NUMERO>
<retroceder>        ->  RETROCEDER <NUMERO>
<girar>             ->  GIRAR <direccion>
<direccion>         ->  DERECHA | IZQUIERDA
<seguir_linea>      ->  SEGUIR LINEA <DURACION>
<levantar>          ->  LEVANTAR <CODIGO_OBJETO>
<mover>             ->  MOVER <COORDENADA>
<soltar>            ->  SOLTAR
<velocidad>         ->  VELOCIDAD <PORCENTAJE>

<repetir>           ->  REPETIR <NUMERO> <lista_instrucciones> FIN
<condicion>         ->  SI OBSTACULO <lista_instrucciones> <parte_sino> FIN
<parte_sino>        ->  SINO <lista_instrucciones>
                        | ε
```

5.4 Propiedades de la gramática

La gramática no tiene recursividad por la izquierda y se puede analizar con un solo token de anticipación. Cada instrucción empieza con una palabra reservada distinta, así que el parser sabe qué regla aplicar con solo mirar el siguiente token.

Tabla 11. Primeros de cada instrucción

Regla	Primeros
avanzar	AVANZAR
retroceder	RETROCEDER
girar	GIRAR
seguir_linea	SEGUIR
levantar	LEVANTAR
mover	MOVER
soltar	SOLTAR
velocidad	VELOCIDAD
repetir	REPETIR

Regla	Primeros
condicion	SI

Dentro de un bloque, el parser sigue leyendo instrucciones mientras el siguiente token sea uno de esos diez. La lista termina cuando aparece FIN o, dentro de un SI, SINO. En el programa principal termina con EOF. Como estos tokens son distintos de los primeros de una instrucción, en cada punto solo hay una opción posible.

Un SINO siempre pertenece al SI abierto más cercano, porque cada SI termina con su propio FIN. Por esa razón la gramática no es ambigua.

5.5 Gramática en ANTLR

RoboScript.g4 contiene la gramática en dos grupos de reglas: las del parser, que empiezan con minúscula, y las del lexer, que van en mayúsculas y se describen en las secciones 3 y 4. Las reglas del parser son las siguientes.

```
programa: robotDecl inicioDecl instruccion* EOF;

robotDecl: ROBOT (CODIGO_ROBOT | CODIGO_ROBOT_INVALIDO | ID)?;

inicioDecl: INICIO COORDENADA?;

instruccion:
    avanzarInst
    | retrocederInst
    | girarInst
    | seguirLineaInst
    | levantarInst
    | moverInst
    | soltarInst
    | velocidadInst
    | repetirInst
    | siInst;

avanzarInst: AVANZAR NUMERO?;

retrocederInst: RETROCEDER NUMERO?;

girarInst: GIRAR (DERECHA | IZQUIERDA | direccionInvalida)?;

direccionInvalida: ID;

seguirLineaInst: SEGUIR LINEA DURACION?;

levantarInst:
    LEVANTAR (CODIGO_OBJETO | CODIGO_OBJETO_INVALIDO | ID)?;

moverInst: MOVER COORDENADA?;

soltarInst: SOLTAR;
velocidadInst: VELOCIDAD (PORCENTAJE | NUMERO | PORCENTAJE_INVALIDO)?;

repetirInst: REPETIR NUMERO? instruccion* FIN;

siInst: SI OBSTACULO instruccion* (SINO instruccion*)? FIN;
```

Estas reglas no son idénticas a la gramática formal de la sección 5.2. En ANTLR varios elementos aparecen como opcionales (por ejemplo AVANZAR NUMERO?) y algunas reglas aceptan además un token de formato incorrecto o un ID. Con esto, cuando falta un elemento el parser igualmente construye el árbol de la instrucción y puede seguir leyendo, de modo que todos los errores del archivo se reportan en una sola ejecución. Después, ResumenListener revisa que cada elemento obligatorio esté presente y genera el mensaje de error (sección 8.3).

Tabla 12. Diferencias entre la gramática formal y las reglas de ANTLR

Elemento	Gramática formal	Regla de ANTLR	Lo valida
Código del robot	<CODIGO_ROBOT>	(CODIGO_ROBOT | CODIGO_ROBOT_INVALIDO | ID)?	ResumenListener
Posición inicial	<COORDENADA>	COORDENADA?	ResumenListener
AVANZAR, RETROCEDER, REPETIR	<NUMERO>	NUMERO?	ResumenListener
Dirección de GIRAR	DERECHA | IZQUIERDA	(DERECHA | IZQUIERDA | direccionInvalida)?	ResumenListener
Duración de SEGUIR LINEA	<DURACION>	DURACION?	ResumenListener
Objeto de LEVANTAR	<CODIGO_OBJETO>	(CODIGO_OBJETO | CODIGO_OBJETO_INVALIDO | ID)?	ResumenListener
Coordenada de MOVER	<COORDENADA>	COORDENADA?	ResumenListener
Porcentaje de VELOCIDAD	<PORCENTAJE>	(PORCENTAJE | NUMERO | PORCENTAJE_INVALIDO)?	ResumenListener
FIN de REPETIR y SI	FIN	FIN	Parser

Un archivo se reporta como válido solo cuando cumple la gramática de la sección 5.2. Los que no la cumplen se reportan con al menos un error.

6. Arquitectura y ejecución

6.1 Estructura del proyecto

Es un proyecto Maven. La gramática está en src/main/antlr4, el código Java en src/main/java y los archivos .robot que usamos para probar el analizador en src/test/resources, separados en dos carpetas: casos y pruebasProyecto.

Estructura de carpetas del proyecto

```
ProyectoAutomatas-roboscript/
  pom.xml
  mvn.bat
  README.md
  src/
    main/
      antlr4/roboscript/
        RoboScript.g4
      java/roboscript/
        Main.java
        errores/
          LexicoErrorListener.java
          SintacticoErrorListener.java
        resumen/
          ResumenListener.java
        util/
          TablaTokens.java
    test/
      resources/
        casos/          (archivos .robot de prueba)
        pruebasProyecto/   (archivos .robot de prueba dados por el Ingeniero)
```

Tabla 13. Clases del proyecto

Clase	Paquete	Función
RoboScriptLexer, RoboScriptParser, RoboScriptBaseListener	roboscript	Las genera ANTLR a partir de RoboScript.g4 al compilar. No se editan.
Main	roboscript	Punto de entrada. Coordina el lexer, el parser, el recorrido del árbol y la impresión del reporte.
LexicoErrorListener	roboscript.errores	Recibe los errores del lexer y los guarda con su línea y columna.
SintacticoErrorListener	roboscript.errores	Recibe los errores del parser, los traduce al español y registra los que detecta ResumenListener.
ResumenListener	roboscript.resumen	Recorre el árbol, cuenta los elementos del programa y valida los elementos obligatorios.
TablaTokens	roboscript.util	Imprime la tabla de lexemas, tokens, líneas y columnas.

6.2 Flujo de ejecución

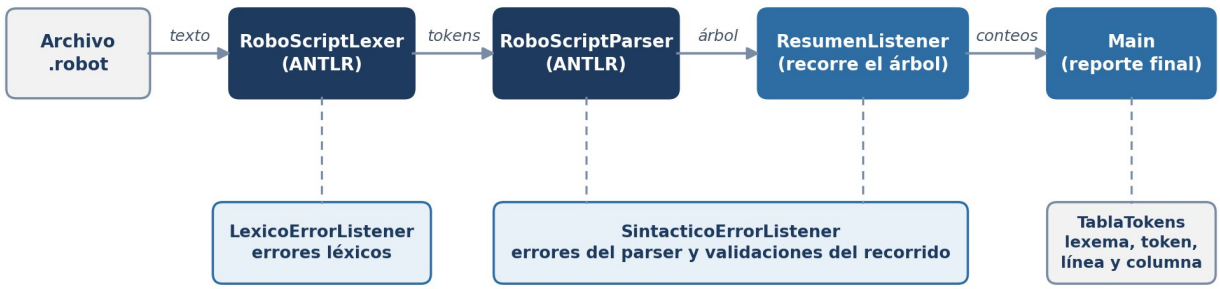


Figura 1. Componentes del analizador y datos que pasan entre ellos.

Cuando se ejecuta el programa, Main sigue estos pasos:

1. Recibe la ruta del archivo .robot como argumento. Si no recibe ninguna, analiza src/test/resources/pruebasProyecto/Prueba 4.robot. Si el archivo no existe, muestra un mensaje y termina.
2. Crea RoboScriptLexer con el contenido del archivo, quita el listener de errores que trae ANTLR por defecto y agrega LexicoErrorListener.
3. Llama a tokens.fill() para que el lexer lea todo el archivo. En este paso se producen los errores léxicos.
4. Imprime la tabla de tokens con TablaTokens y cuenta los tokens reconocidos, sin contar el fin de archivo.
5. Crea RoboScriptParser sobre los tokens, con SintacticoErrorListener, y llama a parser.programa() para obtener el árbol.
6. Recorre el árbol con ParseTreeWalker y ResumenListener, que cuenta las instrucciones y valida los elementos obligatorios.
7. Considera válido el archivo si no hay errores léxicos ni sintácticos, e imprime el reporte correspondiente.

6.3 Compilación y ejecución

Se necesita Java 21 o superior y Maven 3.9 o superior. Desde la carpeta del proyecto:

Comandos para compilar y ejecutar

```
mvn clean package
java -jar target/roboscript.jar src/test/resources/casos/valido_completo.robot
java -jar target/roboscript.jar src/test/resources/casos/invalido_varios_errores.robot
```

En Windows, si Maven no está en las variables de entorno, se puede usar .\mvn.bat clean package. Ese archivo usa el Maven incluido en IntelliJ IDEA cuando encuentra su ruta y, si no, llama a mvn. También se puede ejecutar la clase Main desde IntelliJ IDEA y pasar la ruta del archivo en Run > Edit Configurations > Program arguments.

7. Analizador léxico

7.1 Funcionamiento del lexer

El lexer es la clase RoboScriptLexer, que ANTLR genera a partir de las reglas en mayúsculas de RoboScript.g4. Lee el archivo de izquierda a derecha y, en cada posición, reconoce el texto más largo que cumpla alguna regla. Si dos reglas coinciden con la misma longitud, aplica la que está primero en el archivo (sección 4.2). Cada coincidencia produce un token con su tipo, su texto, su línea y su columna.

Las reglas ESPACIOBLANCO y COMENTARIO terminan con -> skip, así que lo que reconocen se descarta y no genera tokens. Main llama a tokens.fill() para que el lexer lea todo el archivo antes de empezar el análisis sintáctico.

Usamos ANTLR como herramienta de análisis léxico, como pide el enunciado. Las expresiones regulares están dentro de la definición del lenguaje y las usa el lexer generado; el código Java no hace ninguna validación del archivo con Regex.

7.2 Manejo de errores léxicos

Cuando el lexer encuentra un carácter que no inicia ningún token, como @ o $, ANTLR lo descarta, sigue con el resto del archivo y avisa al listener registrado. Main elimina el listener por defecto de ANTLR, que escribe sus mensajes en inglés, y registra LexicoErrorListener.

LexicoErrorListener extiende BaseErrorListener y sobrescribe el método syntaxError. En ese método:

- Extrae el símbolo del mensaje de ANTLR (token recognition error at: '@') y le quita los saltos de línea que el lexer pudo consumir antes de fallar, por ejemplo cuando el símbolo es ! o =, que podrían iniciar != o ==.
- Calcula la columna como charPositionInLine + 1, de modo que empiece en 1, igual que en la tabla de tokens.
- Guarda el mensaje en dos formatos. El formato detallado es ERROR LÉXICO: Línea: 6, Columna: 12, Elemento no reconocido: @. El formato resumido es Línea 6, columna 12: Símbolo no reconocido: @, que es el que usa el reporte final.

Como el lexer continúa después de cada error, todos los símbolos no reconocidos del archivo se reportan en una sola ejecución.

7.3 Tabla de tokens

TablaTokens recorre los tokens y los imprime con su lexema, el nombre del token, la línea y la columna. El nombre sale del vocabulario de ANTLR (RoboScriptLexer.VOCABULARY.getSymbolicName) con el prefijo T_. El token de fin de archivo no se imprime. La tabla se muestra antes del reporte final. El inicio de la tabla de valido_recorrido.robot es el siguiente.

Inicio de la tabla de tokens de valido_recorrido.robot

TABLA DE TOKENS LÉXICOS			
Lexema	Token	Línea	Columna
ROBOT	T_ROBOT	2	1
999XYZ	T_CODIGO_ROBOT	2	7
INICIO	T_INICIO	3	1
X01,Y01	T_COORDENADA	3	8
AVANZAR	T_AVANZAR	6	1
20	T_NUMERO	6	9
GIRAR	T_GIRAR	7	1
DERECHA	T_DERECHA	7	7
AVANZAR	T_AVANZAR	10	1

8. Analizador sintáctico

8.1 Funcionamiento del parser

RoboScriptParser también lo genera ANTLR, ahora a partir de las reglas en minúscula de RoboScript.g4. Es un analizador descendente recursivo: cada regla se convierte en un método (programa(), robotDecl(), instruccion() y así con las demás) que consume tokens y llama a los métodos de las reglas que contiene. Main inicia el análisis con parser.programa() y obtiene el árbol de análisis. Igual que en el lexer, quitamos el listener por defecto y registramos SintacticoErrorListener.

8.2 Errores detectados por el parser

SintacticoErrorListener recibe de ANTLR mensajes en inglés, como missing 'FIN' at '<EOF>' o extraneous input. El método traducirMensaje los convierte en mensajes en español. Para decidir si falta un FIN solo revisa la parte del mensaje que indica qué token se esperaba. Así, un FIN que aparece de más no se confunde con un FIN que falta.

Tabla 14. Mensajes que produce el parser

Situación	Mensaje
Un bloque REPETIR o SI llega al final sin su FIN, o aparece un token que no corresponde dentro del bloque.	Se esperaba FIN.
Aparece un FIN y no hay ningún bloque abierto.	FIN inesperado: no hay un bloque SI o REPETIR abierto.
Aparece un token que no inicia ninguna instrucción, por ejemplo SINO fuera de un SI.	Instrucción o token no esperado: 'SINO'.

Situación	Mensaje
ANTLR menciona PORCENTAJE entre los tokens esperados, por ejemplo después de VELOCIDAD.	Se esperaba PORCENTAJE después de VELOCIDAD.
Cualquier otro caso.	Error en la estructura de la instrucción.

Cuando un bloque queda sin cerrar, el parser llega al final del archivo sin encontrar FIN y reporta el error en la posición del fin de archivo. Si el archivo termina con un salto de línea, esa posición es la línea siguiente a la última instrucción.

8.3 Recorrido del árbol y ResumenListener

ResumenListener extiende RoboScriptBaseListener, la clase que ANTLR genera con un método enter y otro exit para cada regla. Main usa ParseTreeWalker para recorrer el árbol desde la raíz, y cada vez que el recorrido entra en un nodo se ejecuta el método enter de esa regla. ResumenListener solo sobrescribe métodos enter, con dos tareas: contar los elementos que se muestran en el resumen y comprobar que cada instrucción tenga sus elementos obligatorios. Cuando falta alguno, llama a registrarError de SintacticoErrorListener.

Tabla 15. Métodos de ResumenListener

Método	Qué cuenta o guarda	Error que reporta
enterRobotDecl	Guarda el código del robot.	X no cumple con la expresión regular de CODIGO_ROBOT, si el código tiene formato incorrecto o es una palabra suelta. Se esperaba el código del robot después de ROBOT, si falta.
enterInicioDecl	Guarda la posición inicial.	Se esperaba una COORDENADA válida después de INICIO.
enterInstruccion	Suma 1 a las instrucciones analizadas, incluidas las anidadas.	Ninguno.
enterAvanzarInst	Suma 1 a los movimientos.	Se esperaba NUMERO después de AVANZAR.
enterRetrocederInst	Suma 1 a los movimientos.	Se esperaba NUMERO después de RETROCEDER.
enterGirarInst	Suma 1 a los giros.	Se esperaba DERECHA o IZQUIERDA después de GIRAR.
enterLevantarInst	Suma 1 a los objetos levantados.	X no cumple con la expresión regular de CODIGO_OBJETO. Se esperaba CODIGO_OBJETO después de LEVANTAR, si falta.
enterMoverInst	Ninguno.	Se esperaba COORDENADA después de MOVER.
enterSoltarInst	Suma 1 a los objetos soltados.	Ninguno.
enterVelocidadInst	Suma 1 a los cambios de velocidad, solo si viene con porcentaje.	Se esperaba el símbolo '%' después del número en VELOCIDAD, si solo hay un número. Se esperaba número después de VELOCIDAD, si solo hay %. Se esperaba PORCENTAJE después

Método	Qué cuenta o guarda	Error que reporta
		de VELOCIDAD, si falta.
enterSeguirLineaInst	Ninguno.	Se esperaba DURACION después de SEGUIR LINEA.
enterSiInst	Suma 1 a los bloques SI.	Ninguno.
enterRepetirInst	Suma 1 a los bloques REPETIR.	Se esperaba NUMERO después de REPETIR.

Los movimientos suman AVANZAR y RETROCEDER. Una instrucción que tiene el elemento incorrecto, como un AVANZAR sin número, no se cuenta en su contador y se reporta como error.

8.4 Reporte final

Un archivo es válido cuando no tiene errores léxicos ni sintácticos. En ese caso el reporte muestra el análisis léxico (tokens reconocidos y errores léxicos), el análisis sintáctico (instrucciones analizadas y errores sintácticos) y el resumen del programa: robot, posición inicial, movimientos, giros, objetos levantados y soltados, cambios de velocidad, bloques SI y bloques REPETIR.

Si el archivo tiene errores, el reporte indica ARCHIVO NO VALIDO, lista los errores léxicos y los sintácticos y termina con los totales de tokens y de errores. Los errores léxicos se muestran con línea y columna, y los sintácticos con su línea, ordenados por línea. En las secciones 9 y 10 se muestran ejemplos de ambos reportes.

9. Archivos de prueba

Los casos de prueba están en src/test/resources/casos. Son nueve archivos: cuatro válidos y cinco con errores.

Tabla 16. Archivos de prueba

Archivo	Tipo	Qué demuestra
valido_recorrido.robot	Válido	Recorrido en cuadrado con movimientos, giros, levantar, mover y soltar.
valido_completo.robot	Válido	Todas las instrucciones del lenguaje, excepto VELOCIDAD: SEGUIR LINEA, SI, SI con SINO y REPETIR.
cobertura_estructuras.robot	Válido	SI con SINO y un REPETIR que contiene un SI anidado.
extremo_valido_almacen.robot	Válido	Programa largo con varios objetos y bloques SI y REPETIR, incluso anidados.
error_lexico_simbolo.robot	Errores léxicos	Los símbolos @, $ y & mezclados con instrucciones correctas.
error_sintactico_avanzar.robot	Errores sintácticos	AVANZAR y RETROCEDER sin número, y GIRAR

Archivo	Tipo	Qué demuestra
		con una dirección no válida.
invalido_sin_fin.robot	Errores sintácticos	Bloques REPETIR y SI sin FIN.
invalido_varios_errores.robot	Léxicos y sintácticos	Código de robot y de objeto incorrectos, símbolo @, dirección no válida y bloques sin FIN.
extremo_invalido_errores.robot	Léxicos y sintácticos	Cinco símbolos no reconocidos, direcciones no válidas, códigos de objeto incorrectos y bloques anidados sin FIN.

9.1 Archivos válidos

valido_recorrido.robot

Recorre los cuatro lados de un cuadrado y al final levanta, mueve y suelta un objeto.

Contenido de valido_recorrido.robot

```
# Recorrido en cuadrado: avanza por los 4 lados y recoge un objeto al final
ROBOT 999XYZ
INICIO X01,Y01

# Primer lado
AVANZAR 20
GIRAR DERECHA

# Segundo lado
AVANZAR 15
GIRAR DERECHA

# Tercer lado
AVANZAR 20
GIRAR DERECHA

# Cuarto lado, retrocede un poco antes de recoger
AVANZAR 15
RETROCEDER 2

# Recoge y suelta objeto al terminar el recorrido
LEVANTAR OBJ-0042
MOVER X10,Y10
SOLTAR
```

valido_completo.robot

Usa todas las instrucciones del lenguaje, excepto VELOCIDAD, al menos una vez.

Contenido de valido_completo.robot

```
# Prueba valida: usa todas las instrucciones del lenguaje una vez
ROBOT 123ABC
INICIO X10,Y05

# Movimientos basicos
AVANZAR 5
GIRAR DERECHA
AVANZAR 10
RETROCEDER 3
GIRAR IZQUIERDA
```

```
# Manejo de objetos
LEVANTAR OBJ-0025
MOVER X20,Y15
SOLTAR

# Seguir linea con duracion
SEGUIR LINEA 30s

# Condicional simple
SI OBSTACULO
   GIRAR IZQUIERDA
   AVANZAR 5
FIN

# Condicional con SINO
SI OBSTACULO
   RETROCEDER 2
SINO
   AVANZAR 8
FIN

# Repeticion simple
REPETIR 2
   AVANZAR 10
   GIRAR DERECHA
FIN
```

cobertura_estructuras.robot

Combina un SI con SINO y un REPETIR que contiene un SI anidado.

Contenido de cobertura_estructuras.robot

```
# Cobertura de estructuras: anidamiento de SI y REPETIR
ROBOT 789ABC
INICIO X50,Y50

# Movimientos con todas las direcciones
AVANZAR 12
RETROCEDER 3
GIRAR IZQUIERDA
SEGUIR LINEA 45s

# Ciclo de recogida de objetos
LEVANTAR OBJ-9999
MOVER X80,Y90
SOLTAR

# SI con SINO completo
SI OBSTACULO
   AVANZAR 2
   GIRAR DERECHA
SINO
   AVANZAR 8
   GIRAR IZQUIERDA
FIN

# REPETIR con SI anidado adentro
REPETIR 3
   AVANZAR 5
   SI OBSTACULO
      GIRAR DERECHA
      RETROCEDER 1
   SINO
      AVANZAR 3
   FIN
   GIRAR IZQUIERDA
```

```
FIN

# Segunda ronda de movimientos despues del ciclo
SEGUIR LINEA 10s
AVANZAR 4
```

El archivo extremo_valido_almacen.robot, que es más largo, está en el Anexo A.

Tabla 17. Resultado del análisis de los archivos válidos

Archivo	Tokens	Instr.	Mov.	Giros	Lev.	Solt.	SI	REP.
valido_recorrido.robot	25	11	5	3	1	1	0	0
valido_completo.robot	44	18	7	4	1	1	2	1
cobertura_estructuras.robot	52	21	8	5	1	1	2	1
extremo_valido_almacen.robot	173	82	34	24	5	6	3	3

Instr.: instrucciones analizadas. Mov.: movimientos (AVANZAR y RETROCEDER). Lev. y Solt.: objetos levantados y soltados. REP.: bloques REPETIR. En los cuatro archivos los errores léxicos y sintácticos son 0.

El reporte completo de valido_completo.robot es el siguiente.

Reporte de valido_completo.robot

```
==========================================
      ANALISIS DE ROBOSCRIPT
==========================================
Archivo: valido_completo.robot
RESULTADO: ARCHIVO VALIDO
------------------------------------------
ANALISIS LEXICO
------------------------------------------
Tokens reconocidos: 44
Errores léxicos: 0
------------------------------------------
ANALISIS SINTACTICO
------------------------------------------
Instrucciones analizadas: 18
Errores sintácticos: 0
------------------------------------------
RESUMEN DEL PROGRAMA
------------------------------------------
Robot: 123ABC
Posición inicial: X10,Y05
Movimientos: 7
Giros: 4
Objetos levantados: 1
Objetos soltados: 1
Cambios de velocidad: 0
Bloques SI: 2
Bloques REPETIR: 1
==========================================
     ✓ ARCHIVO SINTACTICAMENTE VALIDO
==========================================
```

9.2 Archivos inválidos

error_lexico_simbolo.robot

Tiene tres símbolos que no pertenecen al lenguaje (@, $ y &) y deja instrucciones correctas después de ellos.

Contenido de error_lexico_simbolo.robot

```
# Error lexico: simbolos ilegales mezclados con instrucciones validas
ROBOT 123ROB
INICIO X10,Y05

# El @ y $ son caracteres que el lexer no reconoce
AVANZAR 10 @
GIRAR DERECHA $
RETROCEDER 5 &

# Mas instrucciones validas despues del error, el parser intenta continuar
AVANZAR 3
SOLTAR
```

error_sintactico_avanzar.robot

Tiene AVANZAR y RETROCEDER sin número y un GIRAR con una dirección que no existe.

Contenido de error_sintactico_avanzar.robot

```
# Error sintactico: instrucciones incompletas y direccion de giro no valida
# AVANZAR y RETROCEDER exigen un numero; GIRAR exige DERECHA o IZQUIERDA
ROBOT 123ROB
INICIO X10,Y05

# Error: AVANZAR y RETROCEDER sin numero
AVANZAR
RETROCEDER

# Error: ARRIBA no es DERECHA ni IZQUIERDA
GIRAR ARRIBA

# Instruccion valida despues de los errores
AVANZAR 5
```

invalido_sin_fin.robot

Tiene un REPETIR y un SI que nunca se cierran con FIN.

Contenido de invalido_sin_fin.robot

```
# Error sintactico: bloques REPETIR y SI sin su FIN de cierre
ROBOT 555BOT
INICIO X00,Y00

# Este REPETIR nunca se cierra con FIN, el parser detecta error
REPETIR 4
   AVANZAR 10
   GIRAR DERECHA
   SI OBSTACULO
      RETROCEDER 2
# Faltan dos FIN aqui (uno para SI, uno para REPETIR)
```

invalido_varios_errores.robot

Reúne un código de robot incorrecto, un símbolo no reconocido, una dirección no válida, un código de objeto incorrecto y bloques sin FIN.

Contenido de invalido_varios_errores.robot

```
# Error mixto: combina errores lexicos y sintacticos en el mismo archivo
# Codigo de robot con formato incorrecto (12ROB no cumple el patron NNNLll)
ROBOT 12ROB
INICIO X10,Y05

# Error lexico: @ no es un caracter valido del lenguaje
MOVER X5,Y5 @

# Error de direccion: ARRIBA no existe en la gramatica
GIRAR ARRIBA

# Codigo de objeto invalido: OBJ-25 no tiene los 4 digitos requeridos
LEVANTAR OBJ-25

# Bloques sin su FIN correspondiente (error sintactico)
SI OBSTACULO
   AVANZAR 10

REPETIR 3
   GIRAR DERECHA
```

El archivo extremo_invalido_errores.robot está en el Anexo A.

10. Ejemplos de errores léxicos y sintácticos

10.1 Errores léxicos

El archivo error_lexico_simbolo.robot tiene tres símbolos que el lenguaje no reconoce. El programa los reporta con su línea y su columna y sigue leyendo el resto del archivo. La columna cuenta desde 1: en la línea 6, AVANZAR 10 @, el símbolo @ está en la columna 12.

Reporte de error_lexico_simbolo.robot

```
==========================================
      ANALISIS DE ROBOSCRIPT
==========================================
Archivo: error_lexico_simbolo.robot

RESULTADO: ARCHIVO NO VALIDO
------------------------------------------
ERRORES LEXICOS
------------------------------------------
Línea 6, columna 12: Símbolo no reconocido: @
Línea 7, columna 15: Símbolo no reconocido: $
Línea 8, columna 14: Símbolo no reconocido: &
------------------------------------------
RESUMEN
------------------------------------------
Tokens reconocidos: 13
Errores léxicos: 3
Errores sintácticos: 0
==========================================
```

10.2 Errores sintácticos

En error_sintactico_avanzar.robot, AVANZAR y RETROCEDER no traen el número que exige la gramática y GIRAR ARRIBA usa una dirección que no existe. La instrucción AVANZAR 5 del final es correcta y no genera error.

```
==========================================
        ANALISIS DE ROBOSCRIPT
==========================================
Archivo: error_sintactico_avanzar.robot

RESULTADO: ARCHIVO NO VALIDO
------------------------------------------
ERRORES SINTACTICOS
------------------------------------------
Línea 7: Se esperaba NUMERO después de AVANZAR.
Línea 8: Se esperaba NUMERO después de RETROCEDER.
Línea 11: Se esperaba DERECHA o IZQUIERDA después de GIRAR.
------------------------------------------
RESUMEN
------------------------------------------
Tokens reconocidos: 10
Errores léxicos: 0
Errores sintácticos: 3
==========================================
```

En invalido_sin_fin.robot hay un REPETIR y un SI sin cerrar. El parser llega al final del archivo, que está en la línea 12, y reporta una vez que falta FIN.

```
==========================================
        ANALISIS DE ROBOSCRIPT
==========================================
Archivo: invalido_sin_fin.robot

RESULTADO: ARCHIVO NO VALIDO
------------------------------------------
ERRORES SINTACTICOS
------------------------------------------
Línea 12: Se esperaba FIN.
------------------------------------------
RESUMEN
------------------------------------------
Tokens reconocidos: 14
Errores léxicos: 0
Errores sintácticos: 1
==========================================
```

10.3 Errores léxicos y sintácticos en un mismo archivo

El archivo invalido_varios_errores.robot tiene ambos tipos de error. El reporte los separa en dos secciones.

```
==========================================
      ANALISIS DE ROBOSCRIPT
==========================================
Archivo: invalido_varios_errores.robot

RESULTADO: ARCHIVO NO VALIDO
------------------------------------------
ERRORES LEXICOS
------------------------------------------
Línea 7, columna 13: Símbolo no reconocido: @
------------------------------------------
ERRORES SINTACTICOS
------------------------------------------
Línea 3: 12ROB no cumple con la expresión regular de CODIGO_ROBOT
Línea 10: Se esperaba DERECHA o IZQUIERDA después de GIRAR.
Línea 13: OBJ-25 no cumple con la expresión regular de CODIGO_OBJETO
Línea 21: Se esperaba FIN.
------------------------------------------
RESUMEN
------------------------------------------
Tokens reconocidos: 18
Errores léxicos: 1
Errores sintácticos: 4
==========================================
```

Cada mensaje corresponde a un problema distinto del archivo:

- Línea 3, ROBOT 12ROB: el código tiene dos dígitos y el formato exige tres.
- Línea 7, MOVER X5,Y5 @: el símbolo @ no pertenece al lenguaje. Es un error léxico en la columna 13.
- Línea 10, GIRAR ARRIBA: ARRIBA no es una dirección válida.
- Línea 13, LEVANTAR OBJ-25: el código de objeto tiene dos dígitos y el formato exige cuatro.
- Línea 21: el SI de la línea 16 y el REPETIR de la línea 19 no tienen FIN. El parser lo detecta al llegar al final del archivo.

10.4 Resumen de mensajes

Tabla 18. Errores y mensajes que produce el programa

Tipo de error	Entrada	Mensaje
Símbolo no reconocido	AVANZAR 10 @	Línea 6, columna 12: Símbolo no reconocido: @
Código de robot incorrecto	ROBOT 12ROB	Línea 3: 12ROB no cumple con la expresión regular de CODIGO_ROBOT
Instrucción AVANZAR incompleta	AVANZAR	Línea 7: Se esperaba NUMERO después de AVANZAR.
Dirección de giro no válida	GIRAR ARRIBA	Línea 11: Se esperaba DERECHA o IZQUIERDA después de GIRAR.

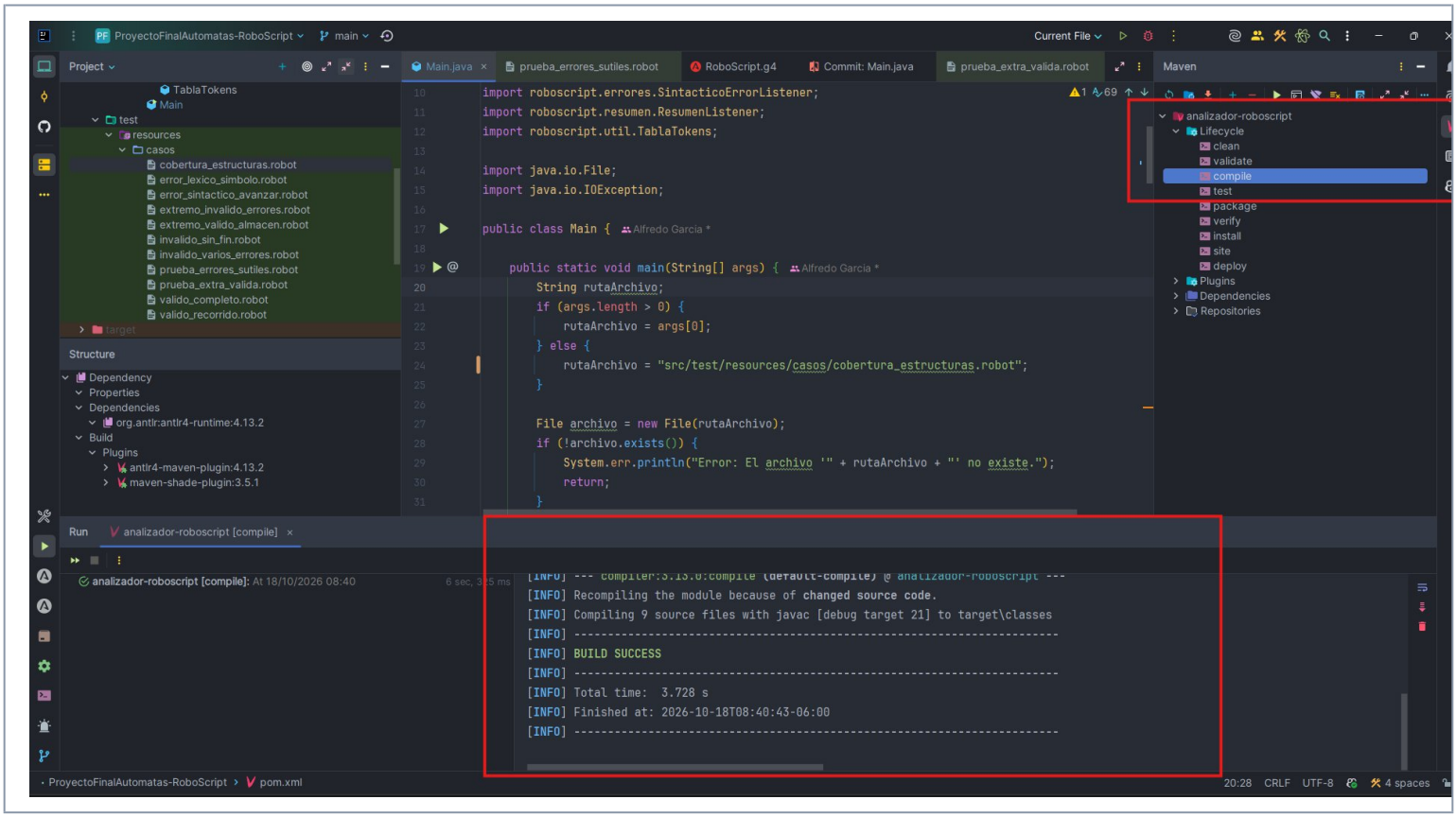
Tipo de error	Entrada	Mensaje
Código de objeto incorrecto	LEVANTAR OBJ-25	Línea 13: OBJ-25 no cumple con la expresión regular de CODIGO_OBJETO
Bloque sin FIN	REPETIR 4 sin cerrar	Línea 12: Se esperaba FIN.

Los números de línea corresponden a los archivos de prueba de las secciones anteriores.

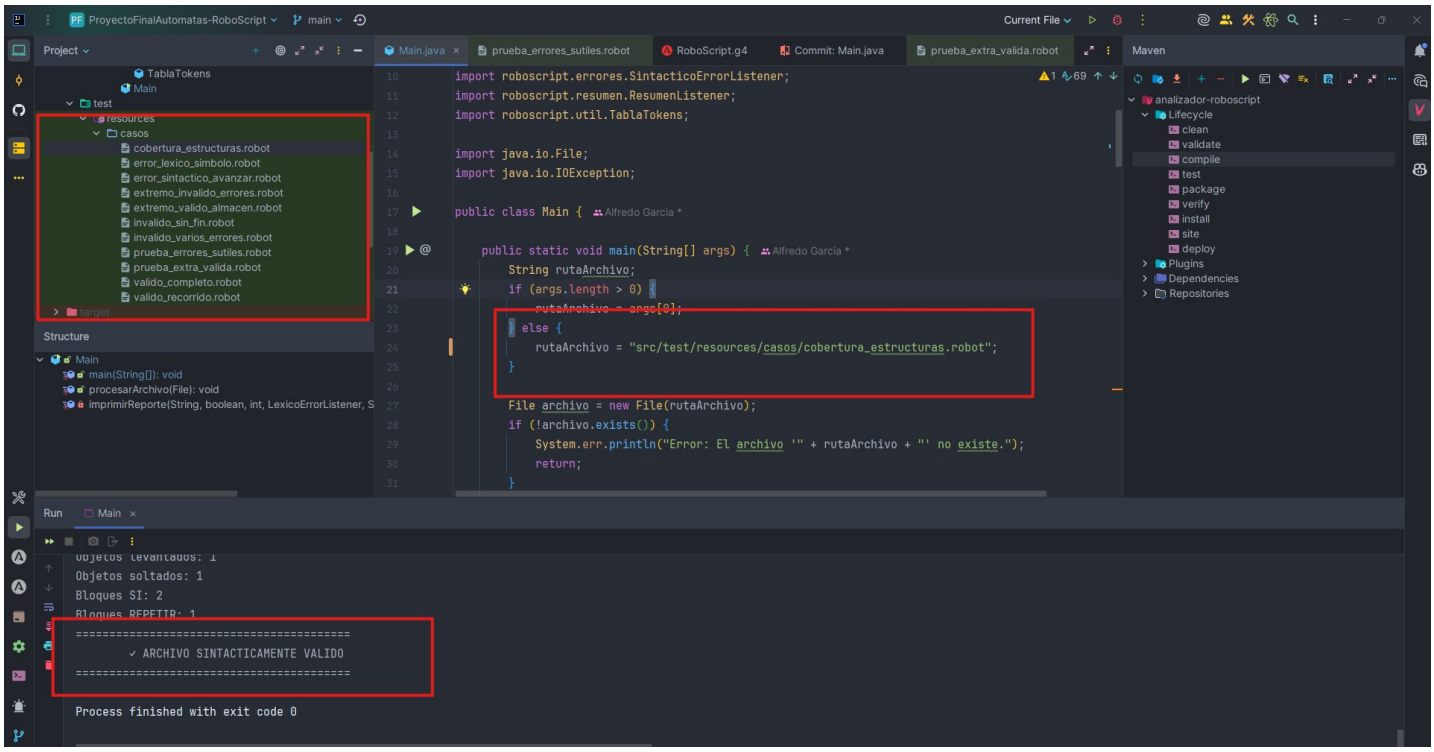
11. Capturas de pantalla

Las capturas muestran el programa en ejecución con un archivo válido y con archivos que tienen errores.

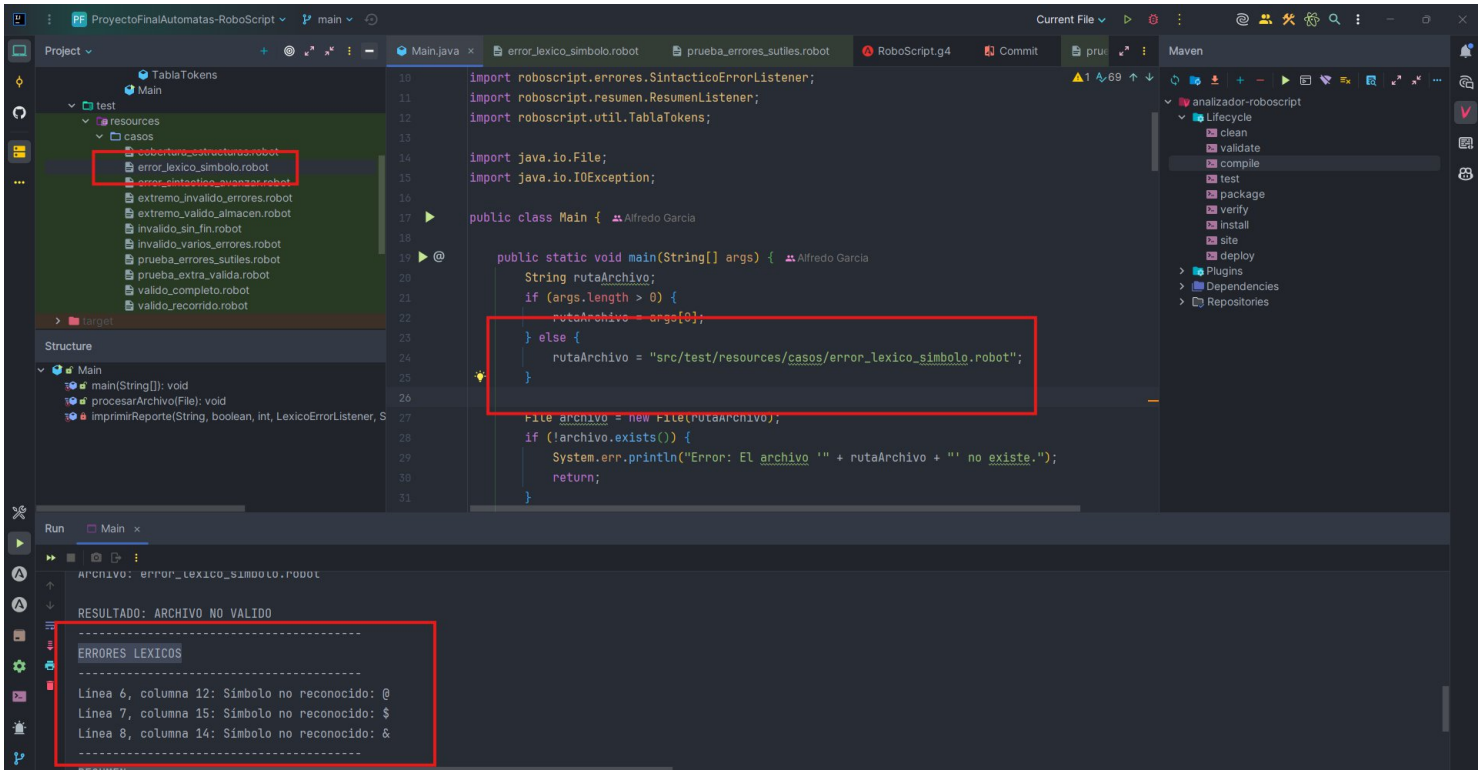
Compilación exitosa en Maven



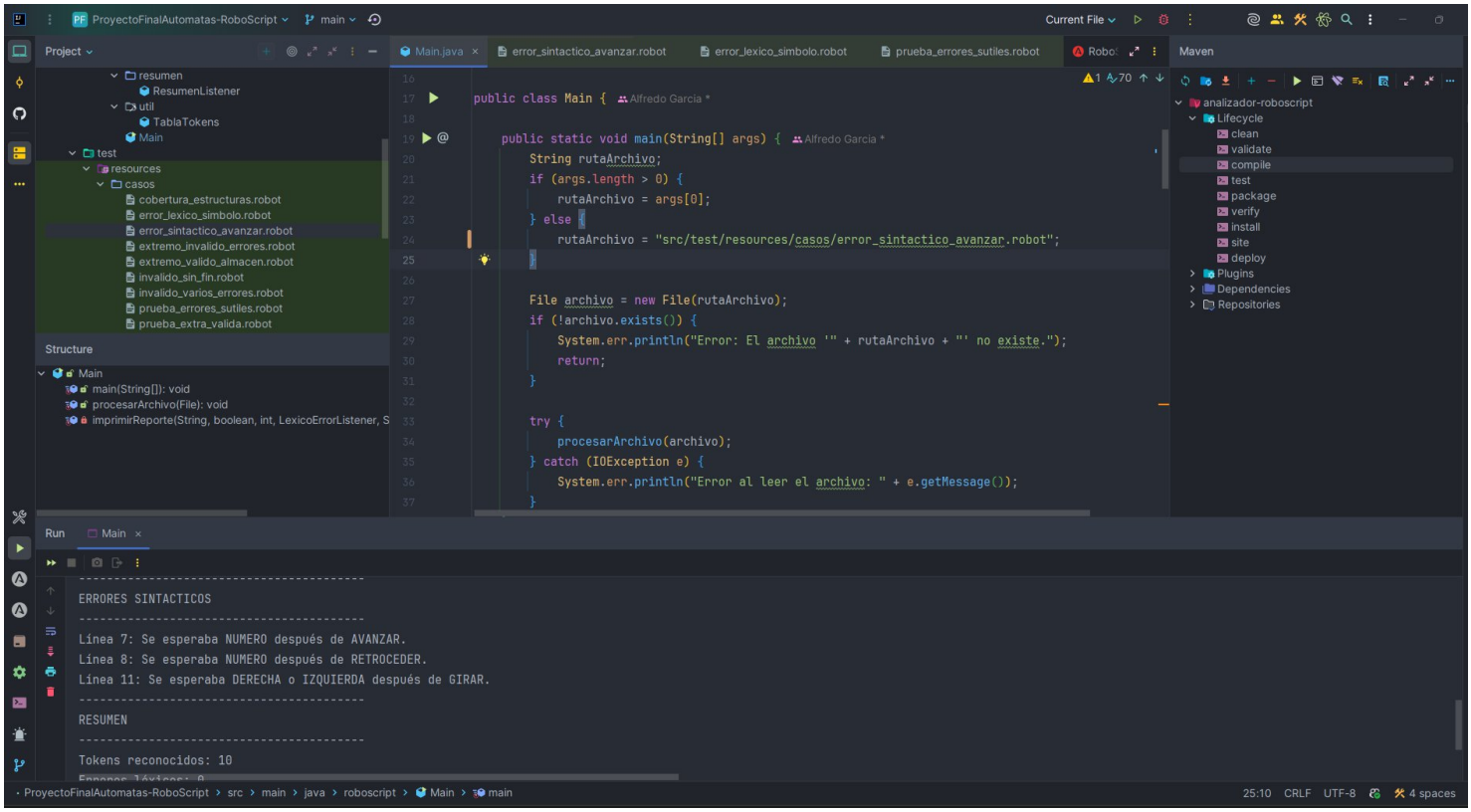
Prueba de archivo valido



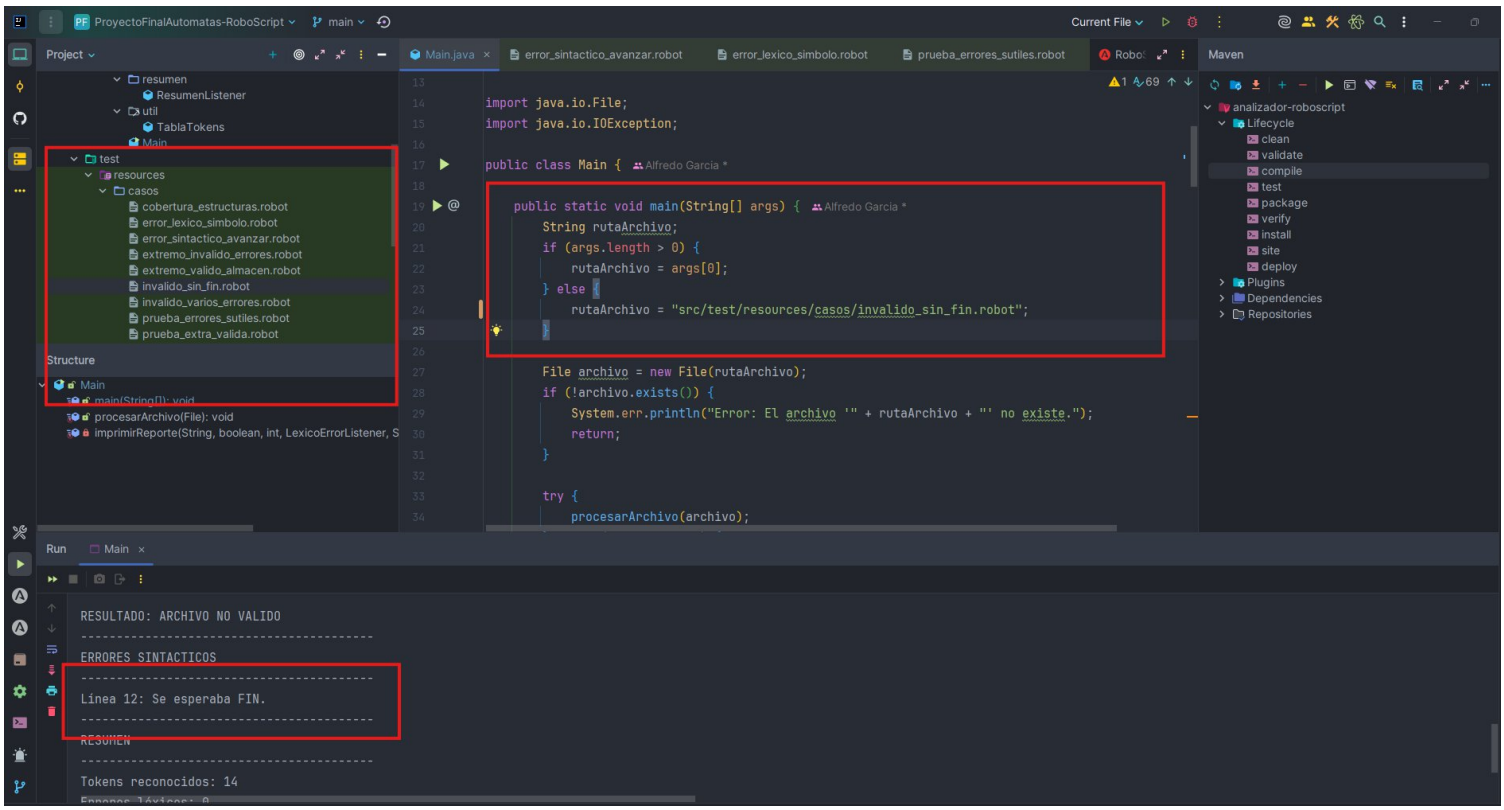
Prueba de archivo con errores lexicos



Prueba de archivo con errores sintácticos



Prueba de archivo con error sin FIN



12. Alcance y decisiones de diseño

Esta sección resume cómo se comporta el programa en los puntos donde el enunciado dejaba margen o donde tomamos una decisión propia.

Tabla 19. Decisiones de diseño

Tema	Decisión
Elementos obligatorios	La gramática de ANTLR deja opcionales varios elementos y ResumenListener comprueba que estén presentes (secciones 5.5 y 8.3).
Códigos con formato incorrecto	Se reconocen con los tokens de formato incorrecto y se reportan con el mensaje X no cumple con la expresión regular de ... . Aparecen en la lista de errores sintácticos y se cuentan en ese total, porque la validación se hace al recorrer el árbol.
Bloques sin cerrar	Si faltan varios FIN, el parser llega una sola vez al final del archivo y reporta un solo error Se esperaba FIN., en la línea donde termina el archivo.
Orden de los errores	Los errores sintácticos se muestran ordenados por línea.
Columnas	Las columnas se cuentan desde 1, tanto en la tabla de tokens como en los errores léxicos.
Contadores del resumen	Los movimientos suman AVANZAR y RETROCEDER. MOVER y SEGUIR LINEA no se cuentan como movimientos. Una instrucción con un elemento incorrecto no se cuenta. VELOCIDAD se cuenta como cambio de velocidad solo si lleva un porcentaje, como 75%.
Comentarios	Agregamos comentarios que empiezan con #. El enunciado no los menciona. Se ignoran por completo.
Formatos de mensaje	Cada listener arma el mensaje en un formato detallado y en uno resumido. El reporte imprime el resumido. El detallado queda disponible en getErroresDetallados().

13. Conclusiones

Desarrollamos un analizador léxico y sintáctico para RoboScript en Java con ANTLR 4. El lexer reconoce las palabras reservadas, los códigos con formato definido, los números y los identificadores, y reporta los símbolos que no pertenecen al lenguaje. El parser verifica la estructura de las instrucciones, incluidos los bloques REPETIR y SI con SINO anidados, y el recorrido del árbol valida los elementos obligatorios de cada instrucción y calcula el resumen del programa.

Probamos el programa con nueve archivos: cuatro válidos y cinco con errores, de los cuales uno tiene solo errores léxicos, dos solo errores sintácticos y dos ambos tipos. El reporte final sigue el formato que pide el enunciado para archivos válidos y no válidos.

Como extensión, la gramática podría incorporar las zonas (CODIGO_ZONA) y los operadores relacionales en las condiciones, que el lexer ya reconoce.

Anexo A. Casos de prueba extensos

A.1 extremo_valido_almacen.robot

Misión de almacén con cinco objetos, varios bloques SI y REPETIR, y un REPETIR que contiene un SI con SINO.

Contenido de extremo_valido_almacen.robot

```
# Prueba extrema valida: misión de almacén con múltiples objetos y rutas largas
# El robot recorre toda una bodega, recogiendo y depositando objetos en zonas
ROBOT 001ABX
INICIO X00,Y00

# ---- ZONA 1: esquina inferior izquierda ----
AVANZAR 10
GIRAR DERECHA
AVANZAR 5
LEVANTAR OBJ-0001
MOVER X15,Y10
SOLTAR

# ---- ZONA 2: desplazarse al siguiente punto ----
AVANZAR 8
GIRAR IZQUIERDA
AVANZAR 12
GIRAR DERECHA
AVANZAR 3

# Chequeo de obstaculo antes de continuar
SI OBSTACULO
   RETROCEDER 2
   GIRAR IZQUIERDA
   AVANZAR 4
SINO
   AVANZAR 6
FIN

LEVANTAR OBJ-0002
MOVER X30,Y20
SOLTAR

# ---- ZONA 3: recorrido en zigzag ----
AVANZAR 10
GIRAR IZQUIERDA
AVANZAR 5
GIRAR DERECHA
AVANZAR 10
GIRAR IZQUIERDA
AVANZAR 5
GIRAR DERECHA
AVANZAR 10

LEVANTAR OBJ-0003
MOVER X55,Y25
SOLTAR

# ---- ZONA 4: ciclo de limpieza del pasillo ----
REPETIR 4
   AVANZAR 5
   GIRAR DERECHA
   AVANZAR 2
   GIRAR IZQUIERDA
FIN
```

```
SI OBSTACULO
   GIRAR DERECHA
   AVANZAR 3
   GIRAR IZQUIERDA
SINO
   AVANZAR 5
FIN

# ---- ZONA 5: seguir linea hacia siguiente zona ----
SEGUIR LINEA 20s
AVANZAR 8

LEVANTAR OBJ-0004
MOVER X70,Y30
SOLTAR

# ---- ZONA 6: ruta larga con repeticion ----
REPETIR 5
   AVANZAR 4
   SI OBSTACULO
      GIRAR DERECHA
      AVANZAR 2
      GIRAR IZQUIERDA
   SINO
      AVANZAR 2
   FIN
FIN

AVANZAR 10
GIRAR DERECHA
AVANZAR 10
GIRAR DERECHA
AVANZAR 10
GIRAR DERECHA
AVANZAR 10

# ---- ZONA 7: deposito final ----
LEVANTAR OBJ-0005
MOVER X90,Y90
SOLTAR

# Volver al origen siguiendo la linea
SEGUIR LINEA 45s

RETROCEDER 5
GIRAR IZQUIERDA
AVANZAR 8
GIRAR DERECHA
AVANZAR 6
GIRAR IZQUIERDA
AVANZAR 10
RETROCEDER 2

# Ciclo de cierre: el robot da vueltas alrededor del punto de inicio
REPETIR 2
   AVANZAR 10
   GIRAR DERECHA
   AVANZAR 10
   GIRAR DERECHA
   AVANZAR 10
   GIRAR DERECHA
   AVANZAR 10
   GIRAR DERECHA
FIN

SOLTAR
```

Reporte de extremo_valido_almacen.robot

```
==========================================
      ANALISIS DE ROBOSCRIPT
==========================================
Archivo: extremo_valido_almacen.robot
RESULTADO: ARCHIVO VALIDO
------------------------------------------
ANALISIS LEXICO
------------------------------------------
Tokens reconocidos: 173
Errores léxicos: 0
------------------------------------------
ANALISIS SINTACTICO
------------------------------------------
Instrucciones analizadas: 82
Errores sintácticos: 0
------------------------------------------
RESUMEN DEL PROGRAMA
------------------------------------------
Robot: 001ABX
Posición inicial: X00,Y00
Movimientos: 34
Giros: 24
Objetos levantados: 5
Objetos soltados: 6
Cambios de velocidad: 0
Bloques SI: 3
Bloques REPETIR: 3
==========================================
     ✓ ARCHIVO SINTACTICAMENTE VALIDO
==========================================
```

A.2 extremo_invalido_errores.robot

Mezcla cinco símbolos no reconocidos, direcciones de giro que no existen, códigos de objeto con menos de cuatro dígitos y bloques REPETIR y SI anidados sin FIN.

Contenido de extremo_invalido_errores.robot

```
# Prueba extrema con errores: múltiples errores lexicos y sintacticos entremezclados
# El objetivo es estresar el recuperador de errores del parser
# Codigo de robot mal formado: 99RX no tiene el formato correcto (NNNLll)
ROBOT 99RX
INICIO X05,Y05

# ---- BLOQUE 1: errores lexicos con simbolos ilegales ----
AVANZAR 10 @
GIRAR DERECHA #este comentario es valido, el # ignora el resto
RETROCEDER 5 $
MOVER X10,Y10 %

# ---- BLOQUE 2: direcciones que no existen en la gramatica ----
GIRAR NORTE
GIRAR SUR
GIRAR ARRIBA
GIRAR ABAJO

# ---- BLOQUE 3: codigos de objeto con formato incorrecto ----
# OBJ-5 y OBJ-99 no tienen los 4 digitos requeridos por CODIGO_OBJETO
LEVANTAR OBJ-5
LEVANTAR OBJ-99
LEVANTAR OBJ-999

# ---- BLOQUE 4: bloques anidados sin su FIN de cierre ----
```

```
REPETIR 3
   AVANZAR 5
   SI OBSTACULO
      GIRAR DERECHA
      REPETIR 2
         AVANZAR 2
         # Este REPETIR interno nunca se cierra

# ---- BLOQUE 5: mas simbolos ilegales intercalados ----
AVANZAR 7 !
RETROCEDER 3 ^
SEGUIR LINEA 10s

# ---- BLOQUE 6: SI sin FIN ----
SI OBSTACULO
   AVANZAR 4
   GIRAR IZQUIERDA
SINO
   RETROCEDER 1
# Falta FIN de este SI

# ---- BLOQUE 7: instruccion valida al final para ver si el parser recupera ----
AVANZAR 1
GIRAR DERECHA
```

Reporte de extremo_invalido_errores.robot

```
==========================================
        ANALISIS DE ROBOSCRIPT
==========================================
Archivo: extremo_invalido_errores.robot

RESULTADO: ARCHIVO NO VALIDO
------------------------------------------
ERRORES LEXICOS
------------------------------------------
Línea 8, columna 12: Símbolo no reconocido: @
Línea 10, columna 14: Símbolo no reconocido: $
Línea 11, columna 15: Símbolo no reconocido: %
Línea 35, columna 11: Símbolo no reconocido: !
Línea 36, columna 14: Símbolo no reconocido: ^
------------------------------------------
ERRORES SINTACTICOS
------------------------------------------
Línea 4: 99RX no cumple con la expresión regular de CODIGO_ROBOT
Línea 14: Se esperaba DERECHA o IZQUIERDA después de GIRAR.
Línea 15: Se esperaba DERECHA o IZQUIERDA después de GIRAR.
Línea 16: Se esperaba DERECHA o IZQUIERDA después de GIRAR.
Línea 17: Se esperaba DERECHA o IZQUIERDA después de GIRAR.
Línea 21: OBJ-5 no cumple con la expresión regular de CODIGO_OBJETO
Línea 22: OBJ-99 no cumple con la expresión regular de CODIGO_OBJETO
Línea 23: OBJ-999 no cumple con la expresión regular de CODIGO_OBJETO
Línea 50: Se esperaba FIN.
------------------------------------------
RESUMEN
------------------------------------------
Tokens reconocidos: 58
Errores léxicos: 5
Errores sintácticos: 9
==========================================
```